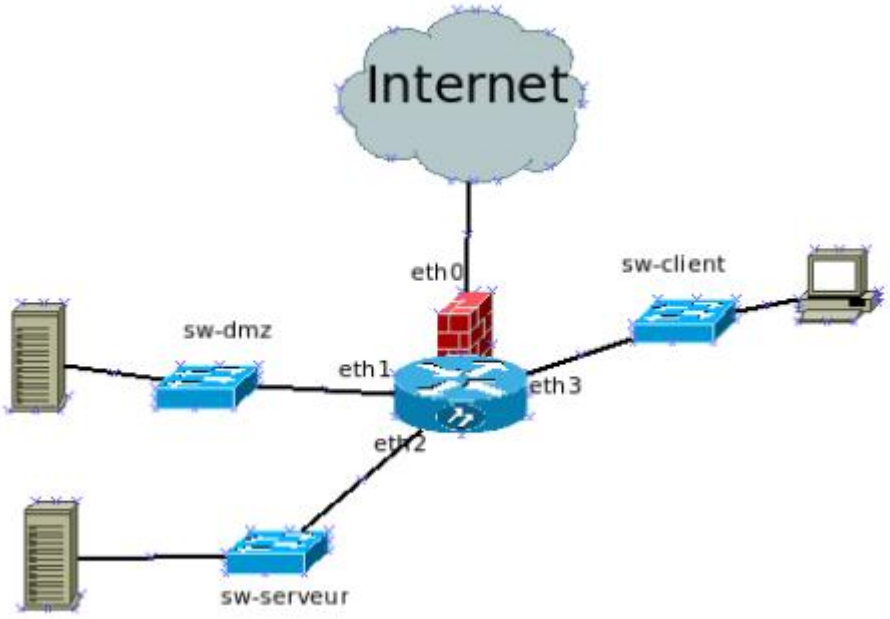
NOM Prénom : Ethan Georget
BTS SIO SISR - Bloc 2
Compte-Rendu : Haute disponibilité de services (Failover Heartbeat & LVS)

DOSSIER TECHNIQUE : HAUTE DISPONIBILITÉ ET RÉPARTITION DE CHARGE

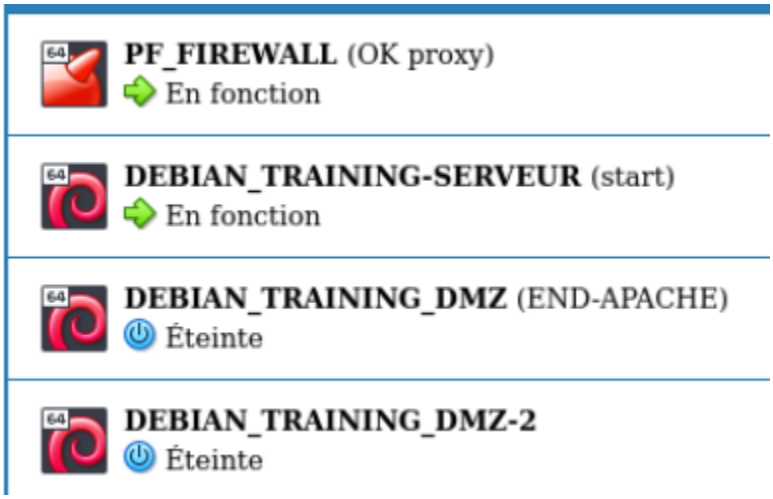
1. Objectifs et topologie du réseau

L'objectif de ce laboratoire est de mettre en œuvre et de tester deux techniques majeures de haute disponibilité (HA) pour un service Web : le Fail-Over (mode Actif/Passif) et le Load-Balancing (mode Actif/Actif). L'infrastructure repose sur le contexte MLIF incluant un pare-feu pfSense, un serveur DNS et deux serveurs Web sous Debian.



1.1 Travaux préparatoires

Deux serveurs Web ont été préparés : srv-web (192.168.200.51) et srv-web-bis (192.168.200.52). Leurs fichiers index.html respectifs ont été modifiés pour identifier clairement le serveur répondant lors des phases de tests. Des instantanés 'apache-base' ont été réalisés pour permettre une restauration rapide entre les deux phases du TP.



1.2 Vérification de la connectivité initiale

Avant de mettre en place la haute disponibilité, j'ai vérifié le bon fonctionnement de la maquette de base. J'ai notamment validé la communication entre les machines et la résolution de noms via le serveur DNS (192.168.100.10) par une série de tests de connectivité (ping).

```
test@client-mlif:~$ ping 192.168.100.10
PING 192.168.100.10 (192.168.100.10) 56(84) bytes of data.
64 bytes from 192.168.100.10: icmp_seq=442 ttl=63 time=0.465 ms
64 bytes from 192.168.100.10: icmp_seq=443 ttl=63 time=0.546 ms
64 bytes from 192.168.100.10: icmp_seq=444 ttl=63 time=0.468 ms
64 bytes from 192.168.100.10: icmp_seq=445 ttl=63 time=0.502 ms
64 bytes from 192.168.100.10: icmp_seq=446 ttl=63 time=0.483 ms
64 bytes from 192.168.100.10: icmp_seq=447 ttl=63 time=0.520 ms
64 bytes from 192.168.100.10: icmp_seq=448 ttl=63 time=0.448 ms
64 bytes from 192.168.100.10: icmp_seq=449 ttl=63 time=0.699 ms
64 bytes from 192.168.100.10: icmp_seq=450 ttl=63 time=0.527 ms
```

2. Haute disponibilité par Fail-Over (Actif/Passif)

Le logiciel Heartbeat est utilisé pour créer un cluster de deux nœuds. Le principe repose sur une adresse IP virtuelle (flottante) qui n'est portée que par le nœud actif. Le nœud passif surveille les battements de cœur (heartbeats) du maître et prend le relais automatiquement en cas de défaillance.

2.1 Configuration du cluster

La configuration repose sur trois fichiers essentiels synchronisés via SCP entre les deux serveurs :
- ha.cf : Définit les paramètres du cluster (deadtime, keepalive et liste des nœuds).
- haresources : Définit le nœud maître et l'adresse flottante (192.168.200.5).
- authkeys : Gère l'authentification sécurisée entre les nœuds (chmod 600 impératif).

- Installer le paquet « heartbeat » ;
- Créer et éditer le fichier « /etc/ha.d/ha.cf » et remplissez-le de la façon suivante :

  #Configuration du cluster
  *#Interface émettant et recevant les battements de cœur : **adapter le nom de la carte !***
  bcast enp0s3
  #Temps, en secondes, avant que le serveur soit déclaré mort
  deadtime 5
  #Intervalle, en secondes, entre deux battement de cœur
  keepalive 1
  #liste des nœuds (serveurs) formant le cluster
  node srv-web-votre_nom srv-web-bis-votre_nom
- Enregistrer le fichier ;
- Créer et éditer le fichier « /etc/ha.d/haresources » et remplissez-le de la façon suivante :

  #Ressources fournies par le serveur actif
  srv-web-votre_nom IPaddr::192.168.200.5 apache2

  Signification des paramètres de ce fichier
  - srv-web-votre_nom → indique le serveur actif par défaut (le maître du cluster) ;
  - IPaddr::192.168.200.5 → Adresse flottante attribuée au serveur actif ;
  - apache2 → Service à lancer sur le serveur lorsqu'il devient actif.
- Enregistrer le fichier ;
- Créer le fichier « /etc/ha.d/authkeys » et remplissez-le de la façon suivante :

  #numéro de méthode d'authentification (1 = 1ère méthode)
  auth 1
  #type de hachage et mot de passe à utiliser pour la méthode 1
  #Le mot de passe utilisé est libre.
  1 md5 motdepasse
- Enregistrer le fichier ;
- Remarquer que dans ce fichier le mot de passe est indiqué en clair. Il faut donc restreindre les droits sur ce fichier au seul utilisateur « root ». Il faut donc lancer la commande :
  chmod 600 /etc/ha.d/authkeys

Synchronisation et sécurité des fichiers de configuration

Après avoir édité les trois fichiers (ha.cf, haresources et authkeys) sur le serveur maître, j'ai utilisé la commande scp pour les copier vers le serveur secondaire afin de garantir une configuration strictement identique sur les deux nœuds. J'ai également veillé à appliquer des droits restrictifs (chmod 600) sur le fichier authkeys pour protéger la clé partagée du cluster.

scp -p root@192.168.200.51:/etc/ha.d/{ha.cf,haresources,authkeys} /etc/ha.d/

2.2 Tests de basculement

Lors de l'extinction brutale du serveur srv-web, l'adresse flottante 192.168.200.5 est automatiquement reprise par srv-web-bis après un délai de 5 secondes. Ce mécanisme garantit une continuité de service transparente pour le client.

Pour confirmer le basculement, j'ai utilisé la commande ip a sur le serveur secondaire après l'extinction du maître. J'ai pu observer l'apparition de l'adresse virtuelle 192.168.200.5 sur l'interface réseau, confirmant que le nœud passif avait correctement pris le relais de manière transparente pour les clients.

3. Haute disponibilité par Load-Balancing (Actif/Actif)

Contrairement au failover, le Load-Balancing permet d'utiliser simultanément tous les serveurs de la grappe. Un répartiteur de charge (Load-Balancer) est intercalé entre les clients et les serveurs réels.

3.1 Configuration de LVS et ipvsadm

Le logiciel LVS (Linux Virtual Server) a été configuré avec l'algorithme Round-Robin (rr). Les requêtes arrivant sur l'adresse virtuelle (VIP) 192.168.200.5 sont redirigées alternativement vers les serveurs réels (192.168.201.11 et 12) via un mécanisme de Masquerading (-m).

ipvsadm -a -t 192.168.200.5:80 -r 192.168.201.11:80 -m
ipvsadm -a -t 192.168.200.5:80 -r 192.168.201.12:80 -m

Pour que la répartition de charge soit persistante, j'ai inscrit les directives dans le fichier /etc/ipvsadm.rules. J'ai ensuite configuré le démon via le fichier /etc/default/ipvsadm pour qu'il se lance en mode "master" au démarrage de la machine, assurant ainsi la pérennité du service de load-balancing.

```
# if you want to start ipvsadm on boot set this to true
AUTO="true"

# daemon method (none|master|backup)
DAEMON="master"

# use interface (eth0,eth1...)
IFACE="enp0s3"

# syncid to use
# (0 means no filtering of syncids happen, that is the default)
# SYNCID="0"
```

3.2 Validation de la répartition de charge

La commande 'ipvsadm -Ln' permet de visualiser l'état de la grappe. Les tests depuis le client confirment que chaque nouvelle requête HTTP est traitée par un serveur différent, équilibrant ainsi la charge de travail.

Une fois les fichiers de règles édités, j'ai utilisé la commande dpkg-reconfigure ipvsadm pour forcer le rechargement de la configuration. Cette étape a permis de valider que les adresses des serveurs réels étaient bien intégrées dans la table de routage du répartiteur de charge avant de lancer les tests de navigation finale.

dpkg-reconfigure ipvsadm

4. Comparaison des solutions

Le Fail-over avec Heartbeat est simple à mettre en œuvre mais laisse un serveur inactif. Le Load-Balancing avec LVS optimise l'utilisation des ressources et offre une meilleure montée en charge.

Dans les deux cas, la haute disponibilité est assurée, supprimant tout point unique de défaillance (SPOF) au niveau des serveurs web.